

Kiste Phase 8 Spec

Self-Hosting, Self-Improving, Modular Bootstrap, Cloud Readiness, and Argo CD Preparation

One-sentence spec: Kiste Phase 8 makes Kiste modular, self-hosting, self-reviewing, and cloud/Argo CD-ready by introducing a small core, loadable modules, staged bootstrap commands, cloud readiness planning, Argo CD manifest generation, and safe self-improvement reports before any automatic production deployment.

Phase	Direction
1	Local Docker repo scanner + token inspection
2	GitHub/Hugging Face scanner + CI
3	Multi-repo workspace graph + token matrix
4	GitOps repo + SOPS-style secret management
4.5	Review, refactor, and architecture checkpoint
5	Python library + CLI
6	Many repo providers + Docker Hub + serverless planning
7	Data repository + dataset intelligence
8	Self-hosting + self-improving + modular bootstrap + cloud/Argo CD preparation

1. Purpose

Phase 8 is the point where Kiste becomes its own first customer. Before Phase 8, Kiste helps other projects. From Phase 8 onward, Kiste should be able to inspect, package, validate, deploy-plan, monitor, review, and improve Kiste itself.

The goal is not uncontrolled self-modification. The goal is a safe loop: observe, analyze, propose, test, review, and only then merge with human approval.

2. Core Principles

Principle	Meaning
Small core, many modules	Kiste core should stay minimal: config, storage, models, module registry, command registry, and r
Bootstrap in layers	Users can start with a minimal setup, then add workspace, GitOps, Argo CD, cloud, data, and self m
Self-improvement by proposal	Kiste may generate reports, issues, patch plans, and draft PRs, but should not auto-merge or auto-
Cloud-ready, not cloud-destructive	Phase 8 prepares plans and manifests. It should not provision or delete cloud resources by default.
Argo CD-ready GitOps	Kiste should generate Argo CD Application, AppProject, and later ApplicationSet YAML from the wo

3. Phase 8 Scope

Included

- Module registry and module graph.
- Staged bootstrap commands.
- Self workspace for Kiste repos.
- Self scan, self check, self review, benchmark, and improvement plan.
- Cloud readiness plan for AWS, GCP, Azure, local Kubernetes, and generic Kubernetes.
- Argo CD Application and AppProject generation.
- GitOps validation and bootstrap reports.
- Safe self-improvement guardrails.

Excluded

- No automatic production deployment.
- No automatic cloud provisioning by default.
- No automatic Argo CD production sync.
- No automatic pruning by default.
- No auto-merge of self-improvement PRs.
- No secret rotation or billing changes without approval.
- No deleting clusters, repos, or production resources.

4. Modular Architecture

Kiste should move toward a small core with loadable modules. The current Python repo already has many features, but Phase 8 should reduce coupling and prevent a single CLI file from becoming the product core.

```

Kiste Core
|
v
Module Registry
|
+-- kiste-repo
+-- kiste-tokens
+-- kiste-workspace
+-- kiste-gitops
+-- kiste-secrets
+-- kiste-images
+-- kiste-targets
+-- kiste-data
+-- kiste-cloud
+-- kiste-argocd
+-- kiste-self

```

Recommended internal module split

```

src/
kiste/
  __init__.py
  cli.py                # thin command wrapper only

kiste_core/
  models.py
  storage.py
  config.py
  module_registry.py
  command_registry.py
  reports.py
  errors.py

kiste_repo/
  scanner.py
  providers.py
  local.py
  github.py
  huggingface.py

kiste_tokens/
  manager.py
  checks.py

kiste_workspace/
  manager.py
  graph.py
  inference.py

kiste_gitops/
  generator.py
  validator.py

kiste_secrets/
  base.py
  sops.py
  scanner.py

kiste_images/
  parser.py
  dockerhub.py
  inspector.py

kiste_targets/
  planner.py
  kubernetes.py
  serverless.py

kiste_data/

```

```

base.py
kaggle.py
roboflow.py
hf_dataset.py
dvc.py

kiste_cloud/
base.py
aws.py
gcp.py
azure.py
kubernetes.py
cluster_plan.py

kiste_argocd/
application.py
app_project.py
appset.py
bootstrap.py
validator.py

kiste_self/
bootstrap.py
review.py
improve.py
benchmark.py
release.py

```

5. Module Manifest

Kiste should track installed and enabled modules in a module manifest. This makes bootstrap easier and prevents the core from loading every provider SDK at startup.

```

{
  "modules": [
    {
      "name": "repo",
      "package": "kiste_repo",
      "enabled": true,
      "commands": ["scan", "repo"],
      "requires": []
    },
    {
      "name": "workspace",
      "package": "kiste_workspace",
      "enabled": true,
      "commands": ["workspace", "connect", "disconnect"],
      "requires": ["repo", "tokens"]
    },
    {
      "name": "gitops",
      "package": "kiste_gitops",
      "enabled": true,
      "commands": ["gitops"],
      "requires": ["workspace", "secrets"]
    },
    {
      "name": "argocd",
      "package": "kiste_argocd",
      "enabled": true,
      "commands": ["argocd"],
      "requires": ["gitops"]
    },
    {
      "name": "cloud",
      "package": "kiste_cloud",
      "enabled": false,
      "commands": ["cloud"],

```

```

    "requires": ["workspace", "gitops"]
  },
  {
    "name": "self",
    "package": "kiste_self",
    "enabled": true,
    "commands": ["self"],
    "requires": ["repo", "workspace", "review"]
  }
]
}

```

Module commands

```

kiste module list
kiste module enable data
kiste module disable data
kiste module doctor
kiste module graph

```

6. Bootstrap Levels

Kiste should support staged bootstrap. This avoids one dangerous giant setup command and lets users adopt only the parts they need.

Level	Name	What it creates
0	Core	kiste.yaml, .kiste/modules.json, base config, storage, models
1	Repo Scanner	local scan state, provider config, repo spec output
2	Workspace	workspace.json, graph.json, token matrix
3	GitOps	gitops.json, secrets.json, GitOps repo skeleton
4	Argo CD	Argo CD Application and AppProject manifests
5	Cloud	cloud-plan.yaml, cluster spec, credential checks
6	Self	self-workspace, self-check, self-review, improvement plan

Bootstrap commands

```

kiste bootstrap core
kiste bootstrap workspace
kiste bootstrap gitops
kiste bootstrap argocd
kiste bootstrap cloud
kiste bootstrap self

```

```

# Or by level
kiste bootstrap --level 3

```

7. Self-Hosting

Self-hosting means Kiste can manage its own lifecycle. It scans its own repos, checks its own tokens, builds its own workspace graph, generates its own GitOps repo, validates its own secrets, and produces its own review and improvement plans.

```

workspace:
  name: kiste-platform
  environment: dev

repos:
  - name: kiste-core
    provider: github
    repo: KisteBox/Python_Repo_Inspector

```

```

    type: backend

- name: kiste-spec
  provider: github
  repo: KisteBox/Kiste_spec
  type: docs

- name: kiste-gitops
  provider: github
  repo: KisteBox/kiste-gitops
  type: infrastructure

- name: kiste-runtime
  provider: github
  repo: KhoaDang-Dinh/kiste
  type: runtime

connections:
- from: kiste-core
  to: kiste-gitops
  relation: generates

- from: kiste-spec
  to: kiste-core
  relation: defines-requirements

- from: kiste-core
  to: kiste-runtime
  relation: controls-runtime

```

Self commands

```

kiste self init
kiste self scan
kiste self check
kiste self review
kiste self improve
kiste self propose --area architecture
kiste self benchmark
kiste self release-plan
kiste self bootstrap --env dev

```

8. Self-Improvement Loop

Self-improvement must be controlled. Kiste should observe its own codebase and CI signals, then propose improvements that are tested and reviewed before merge.

```

Kiste runs checks
-> finds weakness
-> creates improvement report
-> proposes issue / patch / draft PR
-> tests run
-> human reviews
-> merge only with approval

```

Improvement categories

- architecture
- security
- performance
- developer experience
- provider reliability
- test coverage

- documentation
- release quality
- cost/GPU awareness
- GitOps quality

Self-improvement output files

```
.kiste/self/
  self-workspace.json
  self-check.json
  self-review.md
  improvement-plan.md
  benchmark-report.json
  release-plan.md
  cloud-plan.yaml
  argocd-plan.yaml
  bootstrap-report.md
```

9. Cloud-Ready Architecture

Cloud-ready does not mean cloud-deploy yet. In Phase 8, Kiste should prepare cloud integration by generating cloud readiness plans, cluster specs, credential checks, and cost warnings. Real provisioning should remain a later phase unless explicitly enabled by the user.

Cloud providers and targets

- AWS EKS
- Google GKE
- Azure AKS
- generic Kubernetes
- local kind/minikube/dev cluster

Cloud commands

```
kiste cloud plan
kiste cloud plan --provider aws
kiste cloud plan --provider gcp
kiste cloud plan --provider azure
kiste cloud check
kiste cloud credentials check
kiste cloud cluster spec
kiste cloud cost estimate
```

Example cloud cluster spec

```
cloud:
  provider: aws
  kubernetes: eks
  region: ap-southeast-1
  cluster:
    name: kiste-dev
    environment: dev
    node_pools:
      - name: system
        instance_type: t3.medium
        min: 1
        max: 3
      - name: gpu
        instance_type: g5.xlarge
        min: 0
```

```
    max: 2
gitops:
  engine: argocd
  repo: KisteBox/kiste-gitops
```

10. Argo CD-Ready Architecture

Kiste already generates GitOps-style output. Phase 8 should add Argo CD preparation by generating Argo CD Application, AppProject, and later ApplicationSet manifests from the workspace graph.

Argo CD output structure

```
argocd/
  projects/
    kiste-platform-project.yaml
  applications/
    kiste-api.yaml
    kiste-worker.yaml
    kiste-dashboard.yaml
  applicationsets/
    kiste-envs.yaml
```

Argo CD commands

```
kiste argocd init
kiste argocd generate
kiste argocd validate
kiste argocd plan
kiste argocd app list
kiste argocd project generate
kiste argocd appset generate
```

Example Argo CD Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: kiste-api-dev
  namespace: argocd
spec:
  project: kiste-platform
  source:
    repoURL: https://github.com/KisteBox/kiste-gitops.git
    targetRevision: main
    path: apps/api/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: kiste-dev
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
```

Use conservative defaults first: selfHeal: true and prune: false. Automatic pruning can delete resources if the GitOps repo is wrong, so Kiste should not enable pruning by default in Phase 8.

11. Phase 8 End-to-End Flow

```
Kiste CLI
-> Kiste Core
-> Module Registry
-> Workspace Graph
-> GitOps Generator
-> Cloud Cluster Plan
-> Argo CD Application Generator
-> Bootstrap Validation
```


- > Human Approval
- > Later Deployment Phase

Recommended workflow

```
kiste module list
kiste bootstrap core
kiste bootstrap workspace
kiste bootstrap gitops
kiste bootstrap argocd
kiste cloud plan --provider aws
kiste cloud check
kiste argocd generate
kiste argocd validate
kiste self scan
kiste self check
kiste self improve
```

12. Guardrails

Phase 8 should be powerful but safe. Kiste may generate plans, reports, manifests, and proposals. It should not perform dangerous production operations automatically.

Allowed in Phase 8	Not allowed by default
Generate cloud plans	Provision or delete cloud resources automatically
Generate Argo CD YAML	Auto-sync production with pruning enabled
Validate GitOps repo	Auto-merge self-improvement PRs
Check cloud credentials	Rotate production secrets without approval
Create bootstrap reports	Modify billing or account settings
Suggest modular refactors	Delete clusters, repos, or production resources

13. Acceptance Criteria

Phase 8 is complete when Kiste can run the following commands and produce the expected outputs.

```
kiste module list
kiste bootstrap core
kiste bootstrap workspace
kiste bootstrap gitops
kiste bootstrap argocd
kiste cloud plan --provider aws
kiste cloud check
kiste argocd generate
kiste argocd validate
kiste self scan
kiste self check
kiste self improve
```

Expected outputs

- Modular core and module registry.
- Module graph and module doctor output.
- Bootstrap levels for core, workspace, GitOps, Argo CD, cloud, and self.
- Self workspace for Kiste repos.
- Self-check and self-review reports.
- Self-improvement plan.

- Cloud readiness plan and cluster spec.
- Argo CD Application YAML.
- Argo CD AppProject YAML.
- GitOps validation report.
- Bootstrap report with warnings and next steps.

14. Final Phase 8 Direction

From Phase 8 onward, Kiste should become its own first customer. It should use the same commands it gives users - scan, connect, check, review, generate, validate, bootstrap, and improve - on itself.

The architectural goal is a small stable core surrounded by modules. The operational goal is safe self-hosting preparation. The future platform goal is cloud and Argo CD integration without rushing into uncontrolled production deployment.